

Universidad Nacional Autónoma de México

Facultad de Ingeniería

Sistemas Distribuidos

Mexi Banco: que el aviso no tumbe la transferencia
Mensajería asíncrona con RabbitMQ

Bitácora del laboratorio S09

Alumno: Christian Franco
Cuenta: 117003136
Materia: Sistemas Distribuidos
Grupo: Grupo 2
Fecha: 24 de octubre de 2026

Introducción

En el laboratorio de la S08 la transferencia se convirtió en una saga orquestada: si el abono fallaba, el dinero regresaba. Sin embargo, quedaba un cabo suelto crítico: al final de la saga, la transferencia llamaba por HTTP a `notificacion` y *esperaba su respuesta*. Si `notificacion` estaba caído, una transferencia que ya había movido el dinero respondía con error y se quedaba atorada en estado `ABONADA`.

En esta práctica sustituimos esa llamada síncrona (telefónica) por un broker de mensajes (un buzón). La transferencia ahora publica un hecho (`TransferenciaRealizada`) y sigue con lo suyo; `notificacion` lo lee cuando puede. Para ello utilizamos **RabbitMQ**, e implementamos soluciones a los dos problemas clásicos de la mensajería: las entregas duplicadas (resueltas con idempotencia) y los mensajes envenenados (resueltos con una Dead Letter Queue).

1. Parte 0: El problema reproducido

Antes de tocar código, se demostró la falla apagando el servicio de notificaciones y realizando una transferencia por el gateway.

```
docker compose stop notificacion
curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type:
    application/json' -d '{"claveOrigen":"0021800000000000001","claveDestino
    ":"0021800000000000002","monto":50}'
```

La petición devolvió un `HTTP 500 Internal Server Error`. Al consultar las cuentas, los saldos sí se habían movido (el cargo y el abono se aplicaron), pero la transferencia se quedó en estado `ABONADA` en la base de datos.

¿Por qué pasó? La saga ya había movido el dinero (cargo, abono y asientos) y falló al llamar por HTTP a notificación; como esa excepción sale del método antes de `t.marcarCompletada()`, la transferencia se queda a medias y el cliente recibe un error por un aviso que podía esperar.

```
curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json
' \
  -d '{"claveOrigen":"0021800000000000001","claveDestino":"0021800000000000002","monto":50}'

curl -s 'http://localhost:8080/api/transferencias?estado=ABONADA'
curl -s http://localhost:8080/api/cuentas/0021800000000000001
curl -s http://localhost:8080/api/cuentas/0021800000000000002

docker compose start notificacion
[+] stop 1/1
 ✔ Container mexi-banco-v06-notificacion-1 Stopped                                4.6s
HTTP/1.1 500 Internal Server Error
transfer-encoding: chunked
Content-Type: application/json
Date: Sat, 24 Oct 2026 01:58:37 GMT

{"timestamp":"2026-10-24T01:58:37.645+00:00","status":500,"error":"Internal Server Error","
path":"/transferencias"}[]{"id":14,"clabe":"0021800000000000001","titular":"Ana","saldo":100
[+] start 4/4,"clabe":"0021800000000000002","titular":"Beto","saldo":500.00}
 ✔ Container mexi-banco-v06-rabbitmq-1      Healthy                               0.5s
 ✔ Container mexi-banco-v06-db-1            Healthy                               0.5s
 ✔ Container mexi-banco-v06-discovery-1     Healthy                               0.5s
 ✔ Container mexi-banco-v06-notificacion-1  Started                               1.1s
```

Figura 1: Error HTTP

2. Parte 1: Publicar el evento

Para desacoplar la transferencia, se eliminaron las llamadas HTTP a `NotificacionCliente` y se implementó un publicador de eventos en RabbitMQ.

2.1 El publicador y las confirmaciones

Se creó la clase `PublicadorDeEventos`, la cual inyecta un `RabbitTemplate` y publica el evento de forma persistente. Para asegurar que el broker recibió el mensaje, se activaron los *publisher confirms* en el `application.yml`:

```
spring:
  rabbitmq:
    publisher-confirm-type: correlated
    publisher-returns: true
    template:
      mandatory: true
```

El método `publicar()` espera la confirmación (ack/nack) del broker usando un `CorrelationData`. Si el broker no confirma, o si la cola no existe (`mandatory: true` devuelve el mensaje), se lanza una excepción controlada.

El problema de la base de datos

Durante las pruebas, el POST arrojaba un error 500. Al revisar los logs, Hibernate fallaba con un `ConstraintViolationException`. La tabla `transferencia` tenía una columna huérfana llamada `actualizada_en` con un constraint `NOT NULL`, que la entidad ya no mapeaba (Hibernate con `ddl-auto: update` no borra columnas). Se solucionó ejecutando un `ALTER TABLE transferencia DROP COLUMN actualizada_en` directamente en PostgreSQL.

Con notificación apagada, la transferencia ahora responde `201 Created`, se marca como `COMPLETADA` y genera su `eventoId`. El log del servicio indica que el evento no se pudo entregar a ninguna cola (`NO_ROUTE`) porque aún no existía un consumidor, pero la transferencia ya no falla.

```
christian@pc-bda-cfr:~/GodsDocs2/UNIVERSIDAD/9noSemestre/SD/sd-2027-1/entregas/franco_chris
tian/s09/mexi-banco-s09  on 05-mensajeria-base !?
❯ curl -i -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/js
on' \
  -d '{"claveOrigen":"0021800000000000001","claveDestino":"0021800000000000002","monto":30}'
HTTP/1.1 201 Created
transfer-encoding: chunked
Content-Type: application/json
Date: Sat, 24 Oct 2026 03:11:08 GMT

{"id":37,"claveOrigen":"0021800000000000001","claveDestino":"0021800000000000002","monto":30,
"fecha":"2026-10-24T03:11:08.165653435Z","estado":"COMPLETADA","motivo":null,"saldoOrigenTr
asCargo":920.00,"intentosDeCompensacion":0,"actualizada":"2026-10-24T03:11:08.653960749Z","
eventoId":"d77f5f2b-07cf-4e84-81d1-ac554cd326b4"}
christian@pc-bda-cfr:~/GodsDocs2/UNIVERSIDAD/9noSemestre/SD/sd-2027-1/entregas/franco_chris
tian/s09/mexi-banco-s09  on 05-mensajeria-base !?
❯
```

Figura 2: Transferencias

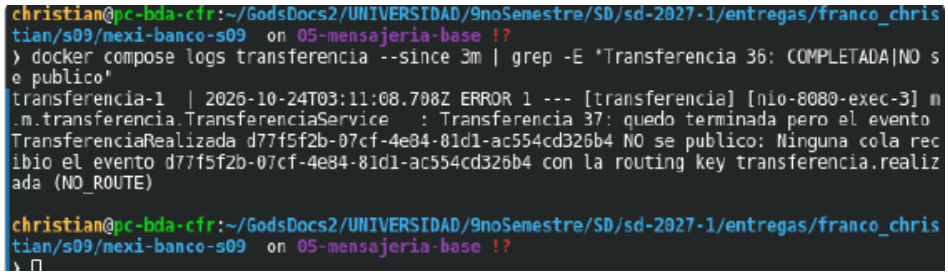


Figura 3: Log evento

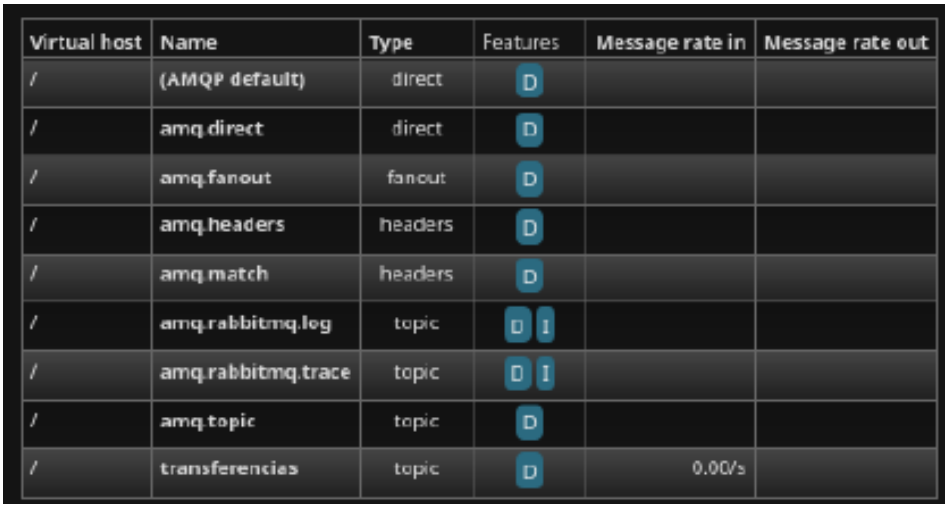


Figura 4: Enchange Panel

3. Parte 2: Consumir el evento

En el servicio de `notificacion` se creó la configuración de la topología (`ConfiguracionMensajeria`) para declarar la cola `notificacion.transferencias` y enlazarla al exchange `transferencias` con el patrón `transferencia.*`.

3.1 Ack manual y desacople en tiempo

Se implementó el `ConsumidorDeTransferencias` con un `@RabbitListener`. Lo más importante es que se configuró el **ack manual** (`acknowledge-mode: manual`) y se programó el `basicAck` para que se ejecute **después** de que los avisos se hayan guardado en la base de datos.

Para demostrar el desacople en tiempo, apagamos `notificacion`, hicimos tres transferencias (quedaron 3 mensajes *Ready* en la cola) y volvimos a encender el servicio. Al arrancar, el consumidor drenó la cola y guardó los 6 avisos (2 por transferencia) en la bandeja de los clientes.

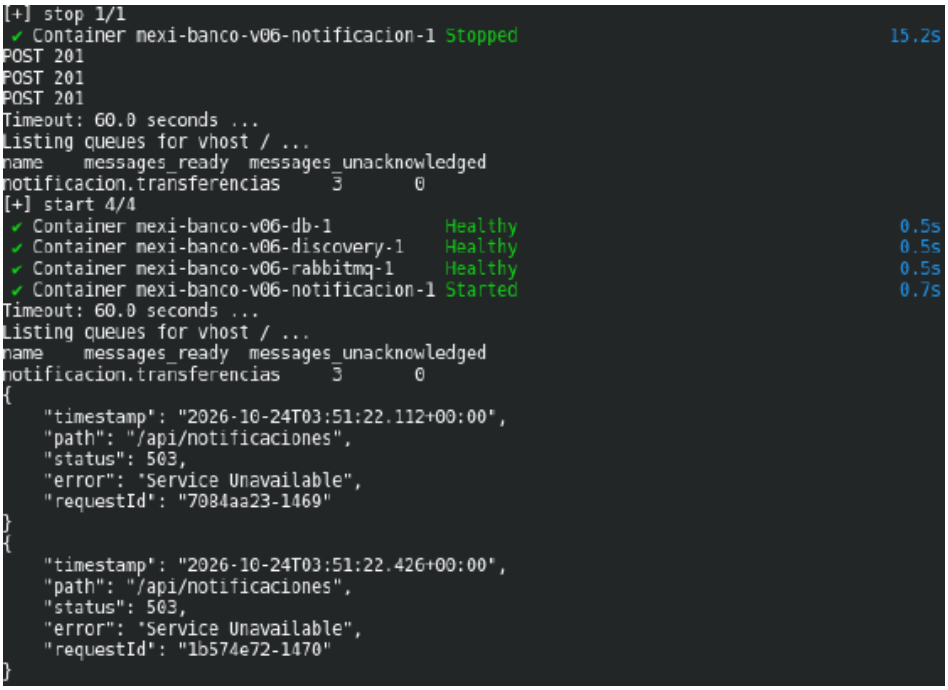


Figura 5: Cola 3 Ready

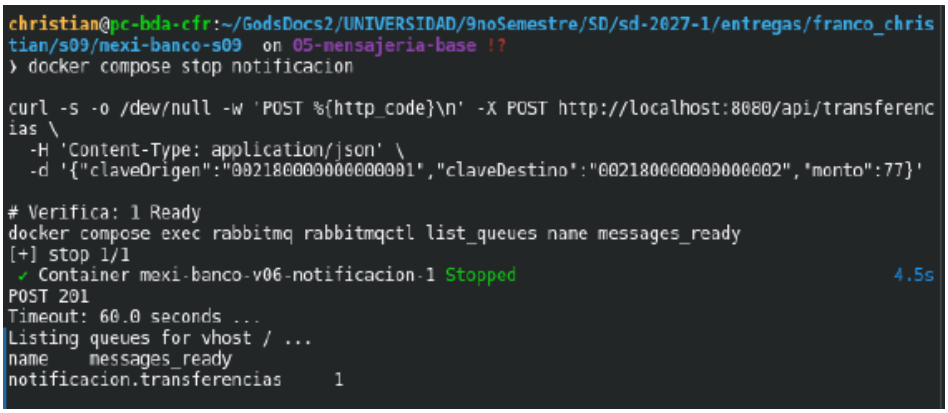


Figura 6: Bandeja Beto

4. Parte 3: El duplicado (Idempotencia)

La mensajería garantiza entrega *al-menos-una-vez*. Si el consumidor muere justo después de guardar los avisos pero antes de hacer el `basicAck`, RabbitMQ volverá a entregar el mensaje, provocando avisos duplicados.

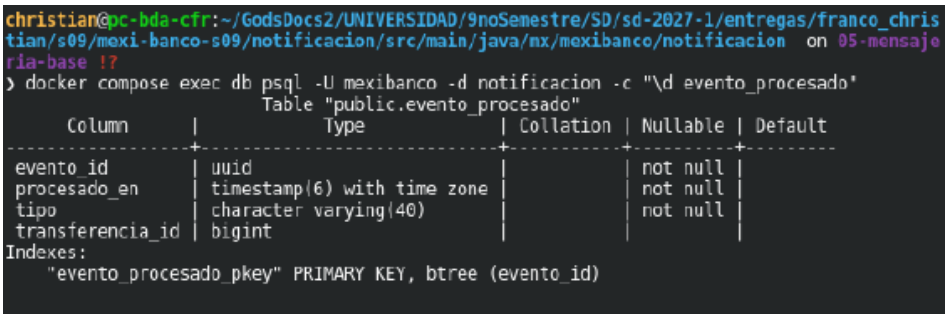


Figura 7: Tabla Evento Procesado

4.1 La solución con ON CONFLICT

Se agregó una tabla `evento_procesado` con el `eventoId` como llave primaria. En el `NotificacionService`, antes de guardar los avisos, se hace un `INSERT ... ON CONFLICT DO NOTHING`.

```
@Modifying
@Query(value = "INSERT INTO evento_procesado (evento_id, tipo, transferencia_id,
    procesado_en) " +
               "VALUES (:eventoId, :tipo, :transferenciaId, now()) " +
               "ON CONFLICT (evento_id) DO NOTHING", nativeQuery = true)
int marcarSiEsNuevo(@Param("eventoId") UUID eventoId, ...);
```

Si la sentencia devuelve `0`, significa que el evento ya se había procesado. El servicio retorna una lista vacía y el consumidor hace `basicAck` sin generar avisos nuevos.

Al forzar el escenario de duplicado, los logs demostraron que la primera entrega insertó los avisos y la segunda imprimió `ya procesado antes; 0 avisos nuevos"`.



Figura 8: P3 log-0-nuevos

5. Parte 4: El mensaje envenenado (DLQ)

Si se publica un mensaje con un JSON inválido, el consumidor lanzará una excepción. Si no se maneja, RabbitMQ reencolará el mensaje infinitamente, atorando la cola (Poison Pill).

5.1 Dead Letter Queue

Se modificó la topología para agregar un Dead Letter Exchange (`transferencias.dlx`) y una Dead Letter Queue (`notificacion.transferencias.dlq`). La cola principal ahora apunta a este DLX. En el consumidor, se agregó un bloque `try-catch`. Si el JSON no se puede leer (`JsonEOFException`), se hace un `basicNack(tag, false, false)` (sin reencolar), lo que hace que RabbitMQ mueva el mensaje automáticamente a la DLQ.

El error PRECONDITION_FAILED

Al intentar arrancar el servicio con la nueva topología, RabbitMQ arrojó `PRECONDITION_FAILED`. Esto ocurre porque no permite cambiar los argumentos de una cola que ya existe. Fue necesario detener el servicio, borrar manualmente la cola `notificacion.transferencias` desde el panel web y reiniciar.

Tras publicar un JSON roto (`{"hola":}`), el log atrapó el error y la cola principal quedó en 0, mientras que el mensaje aterrizó en la DLQ. Al inspeccionarlo, la cabecera `x-death` confirmó el rechazo.

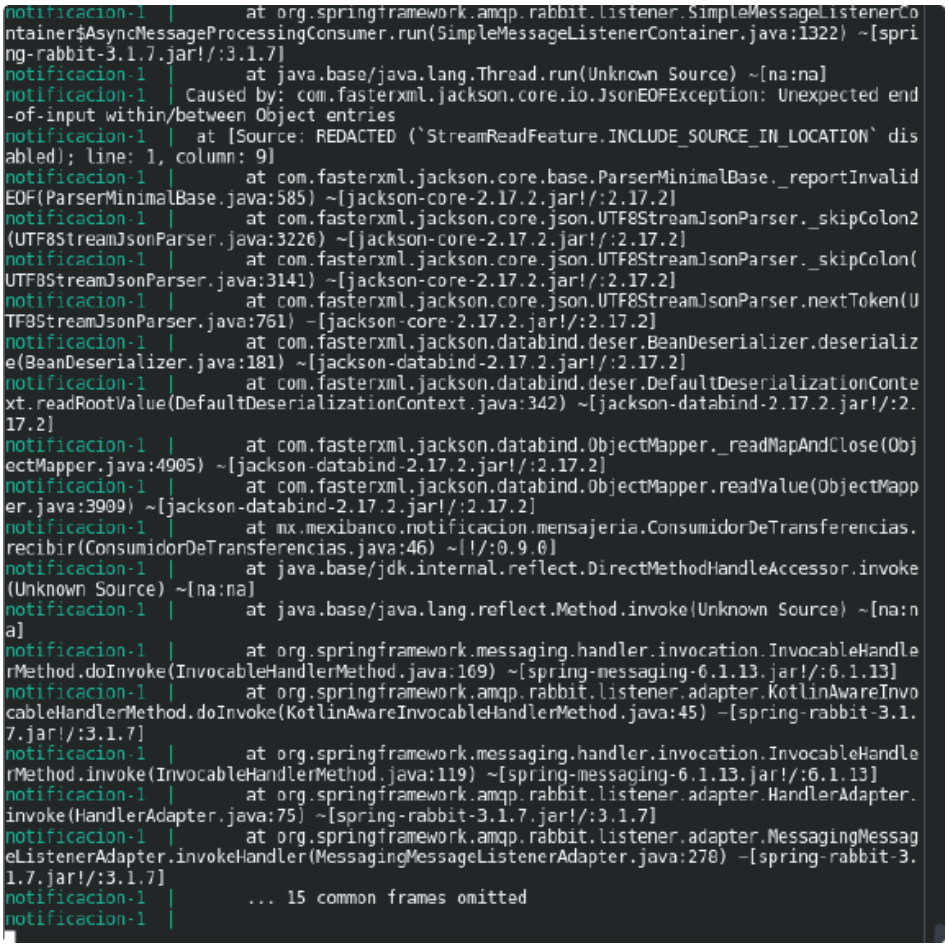


Figura 9: Antes-logs-json

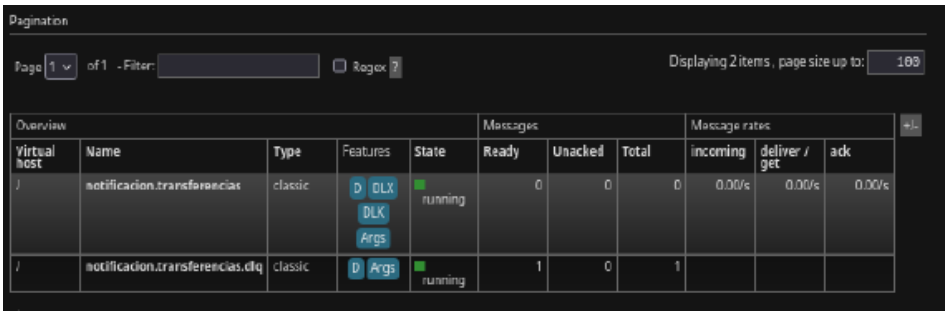


Figura 10: DQL Panel

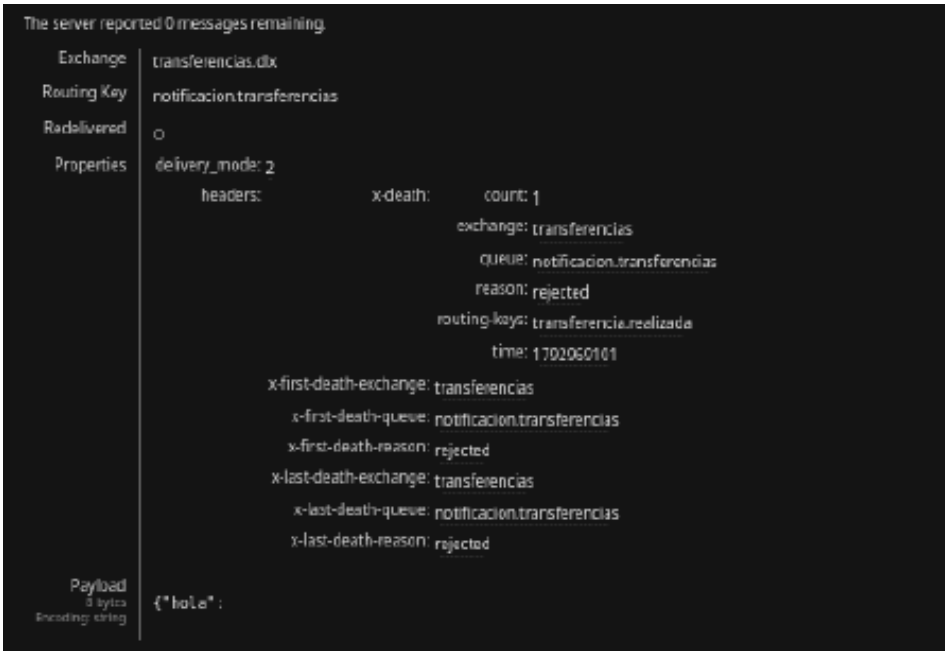


Figura 11: X death

6. Preguntas de cierre

1. Si RabbitMQ está caído cuando la saga llega a COMPLETADA, ¿qué pasa con el aviso en tu versión? ¿Por qué no lo arregla el publisher confirm?
En nuestra versión actual, el método `publicar()` lanza una excepción, pero el bloque `catch` en la transferencia solo registra el error en el log y devuelve un 201 al cliente. El evento se pierde. El *publisher confirm* no lo arregla porque este mecanismo asume que el broker está vivo para responder con un *ack*. Si el broker está caído, no hay nadie del otro lado para confirmar. Para solucionarlo se requeriría el patrón *Transactional Outbox*, que guarda el evento en la misma base de datos local antes de intentar enviarlo.

2. ¿Por qué el ack va después de guardar y no antes? ¿Qué garantía obtienes con cada orden?
Si se hace el ack antes de guardar y el servicio muere en medio, el mensaje se pierde (RabbitMQ lo

borra, pero la base de datos no guardó nada). Al hacerlo después de guardar, si el servicio muere, RabbitMQ no recibe el ack y reencola el mensaje. La garantía que obtenemos es *al-menos-una-vez* (at-least-once), prefiriendo la duplicación de mensajes (que resolvimos con idempotencia) sobre la pérdida de información.

3. Al quitar la llamada síncrona ganaste que el aviso no tumbe la transferencia. ¿Qué perdiste?
Elección: Publicar un evento asíncrono en RabbitMQ en lugar de llamar a notificación por HTTP.
Porque: Un servicio de notificaciones no debe tener el poder de tumbar una transacción financiera que ya se completó.
Sacrifico: La consistencia inmediata y la certeza de entrega en el instante. El sistema ahora es eventualmente consistente; el cliente recibe su 201 antes de saber si el aviso realmente se generó, y dependemos de que el broker no se caiga (o de un Outbox para blindarlo).

Conclusión

Este laboratorio demostró la importancia de la mensajería asíncrona en arquitecturas de microservicios. Pasar de HTTP a RabbitMQ desacopla los servicios y mejora la resiliencia, pero traslada la complejidad al diseño del consumidor: debemos manejar la idempotencia (porque la red no es confiable y habrá duplicados) y las Dead Letter Queues (porque los datos malformados existen). Finalmente, se evidenció que la doble escritura (base de datos + broker) es inherentemente riesgosa sin un patrón adicional como el Outbox, ya que si el segundo paso falla, la información se pierde.